

LeetCode 203: Remove Linked List Elements

Amazon

Arista Networks

Uber

1. Problem Overview & Example

The goal is to remove all nodes from a singly linked list that contain a specific target value (`val`).

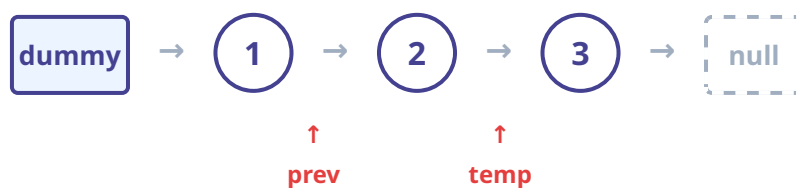
Initial List: `1 → 2 → 3 → 4 → 6 → 7 → 6`

Target Value: `val = 6`

Output List: `1 → 2 → 3 → 4 → 7`

2. Intuition & Logic

To safely delete nodes (especially when the node to be deleted is the head itself), the standard approach is to use a **Dummy Node**. This dummy node acts as an anchor before the actual head of the list.



By maintaining a `prev` pointer starting at the dummy node and a `temp` pointer for traversal, we can always access the node *prior* to the one being deleted. If `temp->val == val`, we adjust `prev->next` to skip the `temp` node entirely.

3. Code Breakdown (Step-by-Step)

Step 1: Handling Initial Edge Cases

If the list is entirely empty, or if it has only one node that happens to be the target value, the code handles a quick exit.

```
if((head==NULL) || (head->next==NULL && head->val==val)) return NULL;
```

Step 2: Dummy Node Setup

A dummy node is allocated with a value of `0` and points to the `head`.

```
ListNode* dummy = new ListNode(0); dummy->next = head;
```

Step 3: Initializing Pointers

Two pointers are created. prev is set to the dummy node (keeping record of the node prior to the one to be deleted), and temp is set to the head for traversing the list.

Step 4: The Traversal Loop (Main Logic)

We loop while temp is not NULL. If the current node's value matches our target (temp->val == val):

- We store the current node in tbd (To Be Deleted).
- We bypass the node by linking prev->next to prev->next->next.
- We move temp forward to evaluate the next node.
- We free the memory using delete(tbd) and continue the loop.

If the value does NOT match, we simply advance both prev and temp pointers.

Step 5: Returning the Result

Because the head of the list might have been deleted, we safely return `dummy->next` which always points to the true start of the modified list.

4. Complete C++ Solution

```
/** * Definition for singly-linked list. * struct ListNode { * int val; *  
ListNode *next; * ListNode() : val(0), next(nullptr) {} * ListNode(int x) :  
val(x), next(nullptr) {} * ListNode(int x, ListNode *next) : val(x),  
next(next) {} * }; */ class Solution { public: ListNode*  
removeElements(ListNode* head, int val) { if((head==NULL) || (head-  
>next==NULL && head->val==val)){ return NULL; } ListNode* dummy=new  
ListNode(0); dummy->next=head; ListNode* prev=dummy; // to keep record of  
node prior to node to be deleted ListNode* temp=head; // to traverse  
while(temp){ if(temp->val==val){ ListNode* tbd=temp; prev->next=prev->next-  
>next; temp=temp->next; delete(tbd); continue; } temp=temp->next; prev=prev-  
>next; } return dummy->next; } };
```